



Unidad Profesional Interdisciplinaria de Ingeniería y
Ciencias Sociales y Administrativas

Trabajo final

Abstracción y uso de datos

Yventz Entzana Garduño

Maya Garcia Aldo

Martinez Perez Alan Eduardo

Reporte de Programas CSV (1 al 8)

Programa 1. Hola Mundo CSV

Objetivo: Aprender a crear y escribir un archivo CSV utilizando C++.

Descripción: El programa genera un archivo llamado hola.csv y almacena el mensaje Hola Mundo.

Entrada: No requiere datos de entrada.

Proceso: Crear el archivo CSV, escribir el encabezado, escribir el mensaje y cerrar el archivo.

Salida: Mensaje / Hola Mundo

Conclusión: Se aprendió a crear y escribir información básica en un archivo CSV.

Programa 2. Suma de dos números CSV

Objetivo: Realizar la suma de dos números y almacenar el resultado en un archivo CSV.

Descripción: El usuario ingresa dos números enteros. El programa calcula la suma y guarda los valores.

Entrada: 10 y 15

Proceso: Leer dos números, calcular la suma y guardar los datos.

Salida: A,B,Resultado -> 10,15,25

Conclusión: Se reforzó el uso de clases y almacenamiento de resultados.

Programa 3. Calculadora Básica CSV

Objetivo: Realizar operaciones aritméticas básicas y guardar los resultados.

Descripción: Calcula suma, resta, multiplicación y división entre dos números.

Entrada: 20 y 10

Proceso: Leer datos, realizar operaciones y guardar resultados.

Salida: Suma=30, Resta=10, Multiplicación=200, División=2

Conclusión: Se aplicaron operaciones matemáticas básicas.

Programa 4. Sobrecarga CSV

Objetivo: Comprender el uso de funciones sobrecargadas.

Descripción: Implementa tres métodos con el mismo nombre, pero diferentes parámetros.

Entrada: No requiere entrada.

Proceso: Ejecutar las tres versiones de la función y guardar resultados.

Salida: 0 parámetros=30, 2 parámetros=15, 3 parámetros=30

Conclusión: Se comprobó el funcionamiento de la sobrecarga.

Programa 5. Herencia CSV

Objetivo: Aplicar el concepto de herencia.

Descripción: Una clase derivada hereda métodos de una clase base.

Entrada: No requiere entrada.

Proceso: Crear clases, ejecutar métodos y guardar resultados.

Salida: Suma=15, Multiplicación=50

Conclusión: La herencia permite reutilizar código.

Programa 6. Herencia y Sobreescritura CSV

Objetivo: Aplicar la sobreescritura de métodos.

Descripción: La clase derivada modifica el comportamiento de un método heredado.

Entrada: No requiere entrada.

Proceso: Sobrescribir el método y ejecutar.

Salida: Resultado=50

Conclusión: La sobreescritura adapta el comportamiento de los métodos.

Programa 7. Nuevo Tipo de Dato Persona CSV

Objetivo: Crear y utilizar tipos de datos definidos por el usuario.

Descripción: Se crea una clase Persona con nombre y edad.

Entrada: No requiere entrada.

Proceso: Crear objeto y guardar datos.

Salida: Juan,20

Conclusión: Se aprendió a modelar información mediante clases.

Programa 8. Promedio CSV

Objetivo: Calcular estadísticas básicas de un conjunto de números.

Descripción: Lee cinco números y calcula suma y promedio.

Entrada: 10,20,30,40,50

Proceso: Leer datos, calcular suma y promedio, guardar resultados.

Salida: Suma=150, Promedio=30

Conclusión: Se reforzó el uso de arreglos y operaciones estadísticas.

1. Merge Sort.

Objetivo

Implementar el algoritmo Merge Sort en C++ para ordenar un arreglo de números enteros y almacenar el resultado en un archivo TXT.

Descripción

Se desarrolló una clase encargada de dividir recursivamente un arreglo y posteriormente fusionar sus elementos en orden ascendente.

Funcionamiento

El arreglo inicial es dividido en partes pequeñas hasta llegar a elementos individuales. Posteriormente se realiza el proceso de combinación ordenada.

Archivo generado: merge.txt

2. Quick Sort.

Objetivo

Ordenar datos utilizando el algoritmo Quick Sort y guardar el resultado en un archivo de texto.

Descripción

Se implementó Quick Sort usando un pivote para reorganizar los elementos del arreglo en menor y mayor valor.

Funcionamiento

El algoritmo selecciona un pivote, divide el arreglo y ordena cada subgrupo recursivamente.

Archivo generado: quick.txt

3. Burbuja indirecta.

Objetivo

Ordenar números mediante método burbuja utilizando punteros indirectos y guardar resultados en TXT.

Descripción

Se trabajó con apuntadores que permiten acceder indirectamente a posiciones de memoria dentro del arreglo.

Funcionamiento

Se comparan elementos consecutivos y se intercambian utilizando referencias indirectas.

Archivo generado: burbujaindirecta.txt

4. Pila Estática.

Objetivo

Implementar una pila estática utilizando arreglos y guardar sus elementos en un archivo TXT.

Descripción

La estructura trabaja bajo el principio LIFO (Last In First Out).

Funcionamiento

Se insertan elementos con push y se administran mediante un índice tope.

Archivo generado: pila.txt

5. Cola Estática.

Objetivo

Implementar una cola estática y guardar su contenido en TXT.

Descripción

La estructura funciona bajo FIFO (First In First Out).

Funcionamiento

Los elementos se insertan al final y se eliminan desde el frente.

Archivo generado: cola.txt

6. Pila dinámica.

Objetivo

Implementar pila dinámica con nodos enlazados y guardar contenido en TXT.

Descripción

La pila usa memoria dinámica mediante punteros.

Funcionamiento

Cada nodo apunta al siguiente elemento y la inserción ocurre en el tope.

Archivo generado: pilaDinamica.txt

7. Cola Dinámica.

Objetivo

Crear una cola dinámica utilizando nodos y almacenar datos en TXT.

Descripción

La cola enlaza nodos dinámicamente.

Funcionamiento

Se insertan nodos al final y se eliminan desde el frente.

Archivo generado: colaDinamica.txt

8. Pila.

Objetivo

Implementar una pila usando STL de C++ y guardar datos en TXT.

Descripción

Se utilizó la biblioteca estándar `stack`.

Funcionamiento

La STL administra automáticamente inserciones y eliminaciones.

9. Merge Indirecto.

Objetivo

Aplicar Merge Sort usando punteros indirectos y guardar resultados en JSON.

Descripción

El algoritmo trabaja manipulando direcciones de memoria.

Funcionamiento

Los valores son ordenados y convertidos a formato JSON.

10. Quick Indirecto.

Objetivo

Aplicar Quick Sort mediante apuntadores y guardar salida en JSON.

Descripción

Se ordena usando referencias indirectas en memoria.

Funcionamiento

Se selecciona pivote y se reorganizan valores.

11. Punteros y referencias

Objetivo

Demostrar uso de punteros y referencias almacenando datos estructurados en JSON.

Descripción

Se utilizaron variables, referencias y direcciones de memoria.

Funcionamiento

Se asignan valores y posteriormente se serializan.

12. Lista Estática.

Objetivo

Guardar una lista implementada con arreglo en formato JSON.

Descripción

La lista administra elementos dentro de un arreglo fijo.

Funcionamiento

Los elementos se insertan secuencialmente.

13. Lista Dinámica.

Objetivo

Implementar lista enlazada dinámica y guardar resultados en JSON.

Descripción

La lista crece dinámicamente mediante nodos.

Funcionamiento

Cada nodo apunta al siguiente elemento.

14. Cola.

Objetivo

Usar la STL queue y almacenar contenido en JSON.

Descripción

Se implementó cola usando biblioteca estándar.

Funcionamiento

La cola administra automáticamente entradas y salidas.

15. Lista

Objetivo

Implementar listas usando STL list y exportar a JSON.

Descripción

Se utilizó la clase estándar list.

Funcionamiento

Los elementos se insertan dinámicamente.

16. ADT.

Objetivo

Implementar un Tipo Abstracto de Dato y guardar información en JSON.

Descripción

Se diseñó una estructura abstracta encapsulando datos y operaciones.

Funcionamiento

El usuario interactúa con métodos sin acceder internamente a la implementación.

Archivo generado:

```
{  
  "dato":25  
}
```

17. Implementación de grafos con almacenamiento TXT

Objetivo

Desarrollar un programa orientado a objetos capaz de representar un grafo mediante una matriz de adyacencia, permitiendo al usuario agregar conexiones entre nodos, visualizar la estructura y almacenar la información en un archivo de texto.

Descripción del programa

El programa implementa una clase llamada Grafo, encargada de representar una estructura de datos no lineal mediante una matriz de adyacencia.

Un grafo es una estructura compuesta por vértices y aristas, donde las conexiones no poseen dirección específica.

El programa permite modificar la estructura dinámicamente desde consola mediante un menú interactivo.

Estructura implementada

Se utilizó la clase: `class Grafo`

Funcionamiento

El programa presenta un menú interactivo:

1. Agregar conexion
2. Mostrar grafo
3. Guardar archivo TXT
4. Salir

18. Implementación de Digrafos con almacenamiento en archivo JSON.

Objetivo

Desarrollar una estructura de datos tipo dígrafo utilizando programación orientada a objetos, permitiendo al usuario crear conexiones dirigidas entre nodos y guardar la información en formato JSON.

Descripción del programa

Un dígrafo (grafo dirigido) es una estructura en la que cada conexión posee una dirección específica.

El programa implementa una clase llamada Digrafo, utilizando una matriz de adyacencia para almacenar relaciones dirigidas.

A diferencia de un grafo tradicional, una conexión:

$A \rightarrow B$

No implica: $B \rightarrow A$

Estructura implementada:

Clase principal: `class Digrafo`

Representacion: `int matriz[4][4];`

19. Árbol binario con almacenamiento en archivo JSON.

Objetivo

Desarrollar un árbol binario utilizando nodos dinámicos y programación orientada a objetos, permitiendo insertar elementos, visualizar la estructura y almacenar la información en un archivo JSON.

Descripción del programa

El programa implementa una estructura jerárquica denominada árbol binario.

Cada nodo contiene:

- un dato
- un nodo hijo izquierdo
- un nodo hijo derecho

La inserción se realiza siguiendo reglas de ordenamiento:

- valores menores a la izquierda
- valores mayores a la derecha

La estructura puede modificarse dinámicamente mediante consola.

Estructuras implementadas

Se desarrollaron dos clases: class Nodo y class Arbol

Cada nodo contiene: int dato;

Nodo* izq;

Nodo* der;

Funcionamiento:

El programa presenta el siguiente menú:

1. Insertar nodo
2. Mostrar arbol
3. Guardar archivo JSON
4. Salir

